

Security Audit — Day 27

1. Objective

The objective of this security audit was to review the workflows developed during Weeks 1–4 and identify potential security issues involving credentials, API keys, webhook access, and customer information.

The workflows were reviewed to ensure that sensitive values are not hard-coded or exposed in workflow files, screenshots, documentation, or GitHub repositories.

2. Findings

Credentials and API Keys

Credentials and API keys should not be hard-coded directly into workflow nodes or JavaScript code.

n8n's credential management system is used to store authentication information securely.

Secrets

Sensitive values such as API keys, passwords, database credentials, and webhook secrets should not be committed to GitHub or included in documentation.

Secrets are kept separately using n8n credentials/variables.

Webhook Security

The existing customer-support webhook was identified as an endpoint that could benefit from an additional security check.

A secret-header validation was added to the webhook.

Customer Data / PII

Some workflows process customer information such as names and email addresses.

Only required information should be processed, and personal information should not be unnecessarily exposed in screenshots, logs, or public repositories.

3. Fixes Implemented

1. Secure Credential Management

- API credentials are stored using n8n's credential system.
- Passwords and API keys are not hard-coded into workflow logic.
- Credentials are not included in exported workflow files intentionally.

2. Webhook Secret Protection

A secret-header check was added to the customer-support webhook.

The request must contain:

X-Webhook-Secret

The expected value is stored securely as the n8n variable:

WEBHOOK_SECRET

The actual secret value is not stored directly in the workflow.

3. Unauthorized Request Handling

The webhook checks whether the provided secret matches the expected secret.

Correct Secret

↓

Continue Workflow

Wrong / Missing Secret

↓

Reject Request

This prevents requests without the correct secret from continuing through the workflow.

4. GitHub Security

Workflow files and documentation were reviewed to ensure that API keys, passwords, database credentials, and webhook secrets are not committed to the repository.

5. Screenshot Security

Sensitive values should not be visible in screenshots submitted as part of the internship deliverables.

4. Security Testing

The secured webhook was tested using Postman.

Test 1 — Correct Secret

The request included the correct X-Webhook-Secret header.

Result: Request was accepted and the workflow continued.

Test 2 — Incorrect Secret

The request included an incorrect secret.

Result: Request was rejected.

Test 3 — Missing Secret

The request was sent without the X-Webhook-Secret header.

Result: Request was rejected.

5. Security Checklist

- Credentials are managed through n8n credentials.
- API keys are not hard-coded into workflow logic.
- Passwords are not committed to GitHub.
- Webhook security was added using a secret header.
- Webhook secret is stored separately as WEBHOOK_SECRET.
- Incorrect webhook secrets are rejected.
- Missing webhook secrets are rejected.
- Secrets are not included in screenshots.
- Secrets are not included in documentation.
- Customer PII is handled carefully.
- Workflow exports are checked before being committed to GitHub.

6. Final Assessment

The Week 1–4 workflows were reviewed for common security issues. Credential handling was kept separate from workflow logic, and an additional secret-header check was added to an existing webhook.

The secured webhook now requires a valid secret before allowing requests to continue, providing an additional layer of protection against unauthorized requests.